TactileSight — Phone-Side Module

What we're building. Qualcomm Snapdragon Multiverse Hackathon, Noida — July 18–19, 2026.

Scope: the **phone** part only. The band (RGB + depth cameras, UNO Q, on-band haptics) is a teammate's scope — see `band-interface.md` for the contract between them.

> **One-liner:** an **on-device, on-demand semantic + multilingual-voice layer** for a blind user. The band handles real-time obstacle avoidance in hardware; the phone answers *"what is this / what does that sign say / what's around me"* — richly, in the user's own language, fused with the band's depth, with nothing leaving the device.

1. Why this wins (benchmark: SixthSense, 2nd place, ExecuTorch hackathon)

SixthSense already shipped our exact domain (phone + haptics for the blind), and it was *good*. We beat/differentiate on **three axes it structurally lacks:**

1. **Richer brain** — Qwen3-VL-4B vs. their Qwen2.5-**0.5B**. Real scene understanding + Q&A + text reading.
2. **Multilingual Indic voice** — AI4Bharat IndicConformer ASR. They have *no* ASR (keyword routing only). Judged in India → this matters.
3. **Depth × language fusion** — band depth zones go *into the VLM prompt* → *"a person is **close** on your left."* Their depth and their LLM never talk; ours do.

Plus a **multi-device "multiverse" story** (head-mounted sensing band → phone) that their chest-strapped-phone + dumb-belt setup doesn't have — aimed straight at Noida's **Multi-Device Innovation** prize.

2. What the phone does (core MVP)

Everything is on-demand. No continuous video feed, no always-on inference — this is the RAM/battery discipline *and* the privacy story.

```
Band button click
      |
      v
Band sends ONE JPEG + depth zones  --(WebRTC)-->  Phone
      |
      v
Gesture:  single = Describe   hold = Query(voice)   double = Save
      |
      v
Qwen3-VL-4B  (depth zones injected into the prompt)   [+ IndicConformer ASR on Query]
      |
      v
```

```
Spoken answer in the user's language  (Android TTS)   +  logged to Scene Memory
```

Interaction (one button, three gestures)

Gesture	Action	Notes
Single click	**Describe**	"What's in front of me?" — fastest, no speaking needed
Hold	**Query** (hold-to-talk)	Ask a spoken question → IndicConformer transcribes → VLM answers
Double click	**Save**	Keep the current frame to the gallery

Band sends **raw button down/up events**; the phone's unit-tested `GestureRecognizer` classifies them.

3. Models & runtimes (deliberately just 2)

Model	Runtime	Job
Qwen3-VL-4B (Q4_0)	**GenieX** (NPU on 8-Elite; llama.cpp GPU/CPU fallback)	Describe / Query / read text (OCR folded in) — **depth zones in the prompt**
AI4Bharat IndicConformer	**sherpa-onnx**	Multilingual Indic speech-to-text (Query)
Android on-device **TTS**	Android	Spoken answer in the user's language

Two inference runtimes (GenieX + sherpa-onnx). Lean on purpose — hackathons are lost on integration, not on missing features. A working 2-runtime demo beats a half-working 3-runtime one.

4. Depth × VLM fusion (our headline differentiator)

Band sends depth as coarse zones with each frame: `{ left: near|mid|far, center: …, right: … }`. We inject them into the VLM prompt:

> *"Depth sensor reports: left=near, center=mid, right=far. Describe what is ahead, using the user's language. Give identity and horizontal position (left/center/right). Be terse."*

The VLM correlates its detected objects with the depth zones itself — **no separate depth model needed** (the band's hardware depth cameras already did the sensing). Result: *"person, left, close."*

5. Supporting features

- **Scene Memory** — 10-min rolling buffer of `{captured frame + VLM description}`. Enables cross-scene queries ("what did that sign 5 minutes ago say?"). Auto-clears. Already built (`SceneMemory.kt`).
- **Language** — user-selectable setting (default: device locale). Demo ships **Hindi + English**.
- **Frame source** — band WebRTC image is **primary**; a **phone-camera + mock-telemetry fallback** is kept and tested (dev unblocker + on-stage demo insurance — WebRTC failing can't kill the demo).

6. Architecture & testing

- **Three seams (interfaces):** `FrameSource`, `SemanticBrain`, `SpeechIO`. Real impls ship; fakes only in tests.
- **Deep module:** `QueryOrchestrator` (gesture → capture → interpret(+depth) → speak → remember).
- **Unified scene state** (`SceneState`-style contract) that components produce/consume — testability parity with SixthSense's clean architecture.
- **Mock mode** — the whole phone stack runs on fakes (phone camera + canned depth) with no band and no models, so everyone can build in parallel.
- **Pure logic is unit-tested** (`GestureRecognizer`, `SceneMemory`, fusion/prompt building) with injected time — no hardware needed to test the brain.

7. Device targets

- **Snapdragon 8 Elite Gen 5** flagship → the real demo (NPU, runs everything full-fat).
- **S22 / S23** → per-component test bench (validate each runtime individually).

8. Stretch (only if the core lands early — do NOT start before the MVP works)

- **YOLOv11n on the clicked frame** → deterministic L/C/R boxes + an instant (~100 ms) first answer while the VLM thinks + boxes on the dashboard.
- **Phone-side visual dashboard** → per-capture view (frame + answer + depth + latency + band battery/connection) for judges / a sighted companion.
- **One opt-in proactive channel** (e.g. "sign ahead: EXIT") — needs a low-rate snapshot stream, which we deliberately dropped to save RAM.

9. Critical-path spikes (these decide everything)

Run on the **S22** as soon as it's connected, in order:

1. **GenieX real VLM inference** — one Describe end-to-end; **measure latency** (is it 1 s or 8 s?).
2. **IndicConformer ASR** — transcribe one Hindi + one English utterance on-device.

3. **Depth-in-prompt** — feed fake zones, confirm the VLM actually uses them.
4. **Glue** — click → {image + depth} → VLM → TTS.

If #1 and #2 both run on-device, we have a winning demo. If either fails, we find out **now**, not on stage.

10. Open risks

- Neither runtime is proven end-to-end yet (**GenieX only compiles**; **IndicConformer is unintegrated**).
- Band↔phone WebRTC link is a cross-team dependency integrated *during* the event → the phone-camera fallback exists precisely for this.